

# Fortran Project - Week 3

## Functions, Subroutines, and File I/O - Final Scientific Data Analyzer

### Today's goal

Refactor the analyzer into reusable functions and subroutines without changing the numerical result, then replace keyboard data entry with file input and save the final analysis results to a file.

### 1. Guided Coding

Study the examples below with your team. Type them into your editor and run them. Do not copy !!

#### [Part A] - Move calculations into functions and subroutines

```
mean = calc_mean(data, n)

contains

double precision function calc_mean(data, n)
  implicit none
  integer, intent(in) :: n
  integer :: i
  double precision, intent(in) :: data(n)
  double precision :: sum

  sum = 0.0d0
  do i = 1, n
    sum = sum + data(i)
  end do

  calc_mean = sum / dble(n)
end function calc_mean
```

#### [Part B] - Replace keyboard input with file input

After the analysis procedures are separated from the main program, the same procedures can be reused when the data source changes from keyboard input to experiment.dat.

```
integer :: i, n
double precision :: x(1000), y(1000)

open(unit=10, file='experiment.dat', status='old')
read(10, *) n

do i = 1, n
  read(10, *) x(i), y(i)
end do

close(10)

print *, 'Number of data = ', n
print *, 'First point = ', x(1), y(1)
```

### 2. Team Coding Tasks

1. Move the mean calculation into calc\_mean().
2. Create calc\_stddev().
3. Create find\_max() and find\_min() as subroutines.
4. Create print\_threshold\_data().
5. Read n and all (x,y) pairs from experiment.dat.
6. Reuse the analysis functions and subroutines on the y array.

7. Print the analysis result on screen.
8. Write the same result to result.dat using WRITE.
9. Include the threshold-selected data in result.dat.
10. Run the complete program from a clean directory containing only the executable and experiment.dat.

### 3. Expected Output / Checkpoint

```
SCIENTIFIC DATA ANALYSIS PROGRAM
Number of data = 10
Mean = ...
Standard deviation = ...
Maximum = ...
Minimum = ...
Results saved to result.dat.
```

The numerical analysis must remain equivalent to the previous version. The main program should now be shorter because calculations are handled by procedures, and the input/output is handled through files.

### 4. Submission Check

- The program compiles without errors.
- The program runs from beginning to end.
- The Week 2 numerical results are preserved after modularization.
- The program reads experiment.dat and creates result.dat.
- Every team member can explain the functions, subroutines, and file I/O added this week.
- Save the completed source code as week3\_project.f90.